

hanford

2025年12月

文档下载

一	二	三	四	五	六
1	2	3	4	5	6
8	9	10	11	12	13
15	16	17	18	19	20
22	23	24	25	26	27
29	30	31	1	2	3
4	5	6	7	8	9
10					

昵称: hanford
园龄: 9年1个月
粉丝: 41
关注: 0
+加关注

搜索

找找看

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签
更多链接

我的标签

VC++(10)
串行通讯(4)
博客(2)
Word(2)
面积(1)
静态库(1)
自动唤醒(1)
编译(1)
睡眠(1)
流控制(1)
更多

随笔分类

GPS(1)
Math(6)
Office(3)
VB(1)
VC++(25)
Windows(12)
测绘(19)
串行通讯(5)
计算器(4)
网络(2)
物理(14)
字符串(6)

随笔档案

2017年1月(9)
2016年12月(74)
2016年11月(30)

阅读排行榜

1. Ntrip通讯协议1.0(20007)
2. 点位精度评定(16386)
3. 内联函数(15202)
4. Windows文本文件编码(12830)
5. Windows高精度时间(11828)

评论排行榜

1. 串行通讯之.NET SerialPort异步写数据(14)
2. 缓和曲线03三次抛物线(5)
3. 缓和曲线09正弦一波型(4)
4. 子午线弧长(3)
5. Windows代码页、区域(2)

推荐排行榜

Windows代码页、区域

目录

第1章代码页 1

1 代码页 1

1.1 单字节字符集 1

1.2 双字节字符集 1

1.3 多字节字符集 1

1.4 ANSI代码页 2

2 枚举代码页 3

3 查询代码页信息 3

4 宽窄字符串 4

5 字符串转换 5

5.1 查表 5

5.2 NlsDllCodePageTranslation 6

第2章区域 8

2.1 一个例子 8

2.2 setlocale 9

2.2.1 简单用法 9

2.2.2 复杂用法 9

2.3 #pragma setlocale 10

2.4 rc 文件 11

2.5 排序 12

第1章代码页

1 代码页

代码页也叫字符集，它有两个特点：

- 1、它是一个字符集合；
- 2、为了便于计算机处理。这个字符集合里，每个字符都有编码。

可用一个字符串表示代码页，如：GB2312、GBK、GB18030、Big5……也可以用一个整数表示代码页，如：20936表示GB2312、936表示GBK、54936表示GB18030、950表示Big5……

1.1 单字节字符集

代码页里，每个字符使用一个字节编码，这样的字符集就是单字节字符集SBCS（Single-byte Character

1. Ntrip通讯协议1.0(6)
2. Windows 位图(2)
3. 串行通讯之.NET SerialPort (2)
4. 点位精度评定(1)
5. VC++编译zlib(1)

评论

- 📄 文档下载
- Re:libCEF总结02字符串
Re:cef_string_utf16_set方法
Re:cef_string_t没用, 还是
方法有用!
--整鬼专家
- Re:缓和曲线03三次抛物线
次抛物线并未广泛应用于高速
铁路, 而是应用于早期低速铁
路。随着CAD技术发展, 即使
低速铁路也不用三次抛物线了,
都是使用回旋线。
--风一般的男人
3. Re:缓和曲线03三次抛物线
非常感谢!
--ymd123456
4. Re:缓和曲线03三次抛物线
@ymd123456 请看#2楼回复...
--hanford
5. Re:缓和曲线03三次抛物线
--hanford

Sets)

1.2 双字节字符集

代码页里, 每个字符最多使用两个字节编码, 这样的字符集就是双字节字符集DBCS (Double-byte Character Sets)

1.3 多字节字符集

代码页里, 某些字符的编码超过了一个字节, 这样的字符集就是多字节字符集MBCS (Multi-byte Character Sets)。显然, 双字节字符集属于多字节字符集, 反过来多字节字符集不一定是双字节字符集。因为, 有些代码页会用两个以上的字节表示一个字符。如: UTF-7、UTF-8……。

笔者发现一个规律: Windows中, 编码超过两个字节的代码页, 其数值超过50000, 如下图所示:

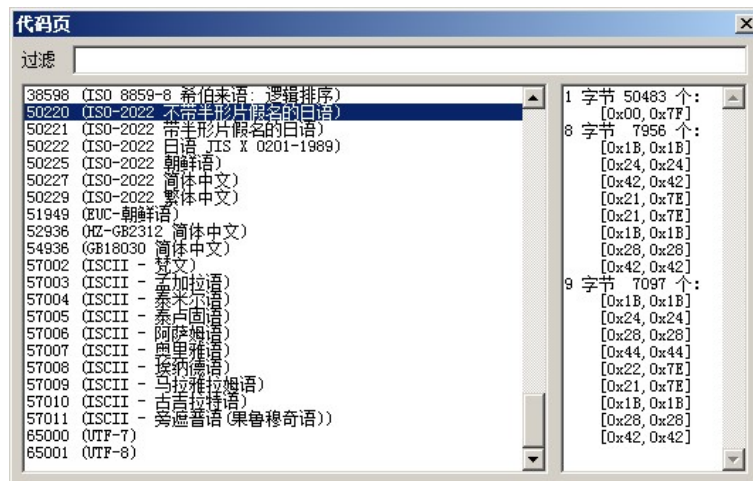


图1.1

上图中, 除了代码页“51949 (EUC-朝鲜语)”, 剩下超过50000的代码页, 其编码用到的最大字节数均大于二。

1.4 ANSI代码页

ANSI代码页具有如下特点:

- 1、编码0至127符合ANSI (American National Standards Institute——美国国家标准学会) 制定的ASCII编码标准;
- 2、它是由微软制定并实现的。如: GB2312也符合第1条, 但它不是ANSI代码页, 因为这套编码属于中国的国标, 不是微软制定的。由微软实现的GBK才是ANSI代码页;
- 3、它是双字节字符集, 亦即编码最多两个字节。

下图中, 有ANSI标志的就是ANSI代码页。简体中文Windows, 使用的代码页是936, 即GBK。

874	(ANSI/OEM - 泰语)
875	(IBM EBCDIC - 现代希腊语)
932	(ANSI/OEM - 日语 Shift-JIS)
936	(ANSI/OEM - 简体中文 GBK)
949	(ANSI/OEM - 朝鲜语)
950	(ANSI/OEM - 繁体中文 Big5)
1026	(IBM EBCDIC - 土耳其语 (拉丁语-5))

图1.2

2 枚举代码页

可使用API函数EnumSystemCodePages, 枚举系统的代码页。下面的代码枚举代码页, 存入变量s_mapCodePage中:

```
static std::map<UINT,CString> s_mapCodePage;
```

```
static BOOL CALLBACK EnumCodePagesProc(LPTSTR lpString)
{
    s_mapCodePage[_tcstoul(lpString,NULL,10)];

    return TRUE;
}

{//枚举代码页，存入s_mapCodePage

s_mapCodePage.clear();

EnumSystemCodePages(EnumCodePagesProc,CP_INSTALLED);

}
```

3 查询代码页信息

可使用GetCPInfoEx函数获得代码页的信息。代码如下：

```
//遍历s_mapCodePage，获取每个代码页的说明
CPINFOEX ci;

for(std::map<UINT,CString>::iterator it = s_mapCodePage.begin()
;it != s_mapCodePage.end();++it)
{
    if(GetCPInfoEx(it->first,0,&ci))
    {
        it->second = ci.CodePageName;
    }
}

}
```

结构CPINFOEX里，多字节编码的信息不全——只有首字节的范围信息，没有其它字节的范围信息。

可以通过编码找出其余字节的范围信息。思路就是：调用WideCharToMultiByte函数，将0~0xFFFF的UTF-16编码转换为指定代码页的编码，并确定各个字节的范围。如下图所示，计算出了GBK的编码范围：



图1.3

注意：

- 1、上图中，一字节的编码范围为[0x00,0xFF]，最多只能有256个。何以有41198个之多？原因在于WideCharToMultiByte将UTF-16字符映射为GBK字符时，会有多个字符映射为同一个字符的情况；
- 2、上图的编码范围只显示最小值和最大值，还不足够精细；

3、实现上述功能的VC++代码已被笔者上传至git服务器，网址如下：

<https://github.com/hanford77/Exercise>

<https://git.oschina.net/hanford/Exercise>

在工程WinNLS里。

4 宽窄字符串

Windows 下，使用VC++编程。会遇到三类字符串：

1、宽字符串，即UTF-16编码的字符串。每个字符固定占用两个字节。处理宽字符串的API函数一般以W结尾，如：CreateWindowW、MessageBoxW……

2、窄字符串，即字符串中的字符均属于某个ANSI代码页。每个字符占用一至两个字节。处理窄字符串的API函数一般以A结尾，如：CreateWindowA、MessageBoxA……

3、多字节字符串，即字符串中字符编码的字节数超过了两个。如：UTF-8、GB18030……这类字符串必须转换为宽字符串或窄字符串后，才能被Windows API使用。

Windows系统中，宽字符串不会产生歧义——它总是UTF-16编码；多字节字符串（包括窄字符串）在不同的代码页下会有不同的解释，所以必须明确窄字符串所属的代码页，否则就会产生乱码。

5 字符串转换

宽窄字符串的转换由WideCharToMultiByte（宽字符串转换为多字节字符串）和MultiByteToWideChar（多字节字符串转换为宽字符串）完成。

5.1 查表

WideCharToMultiByte和MultiByteToWideChar的实质工作主要就是查表。如下面的代码转换宽字符串“编码”为窄字符串：

```
char szStr[64];

WideCharToMultiByte(CP_ACP,0,L"编码",-1,szStr,64,NULL,NULL);
```

这一行代码做了什么？查看注册表

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Nls\CodePage

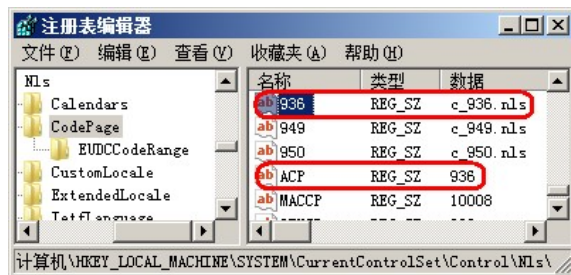


图1.4

根据上图可知：CP_ACP表示代码页936。再根据936可以查到文件c_936.nls。nls文件其实就是UTF-16编码与ANSI编码的对照表。具体请参考博文：

<http://demon.tw/copy-paste/nls-file-format.html>

WideCharToMultiByte根据c_936.nls文件中的对照表，把“编码”由UTF-16编码转换为代码页为936的编码。

5.2 NlsDllCodePageTranslation

有些编码不适合查表，如：UTF-7编码与UTF-16编码的转换并不是简单的字符对应关系，无法查表完

成。

编码超过两个字节的，无法查表。如：GB18030、UTF-8的编码均超过了两个字节，无法使用nls文件存储编码对照表。

这类情况下，WideCharToMultiByte和MultiByteToWideChar是如何实现的呢？查看注册表，代码页54936（GB18030）对应的文件是c_g18030.dll。

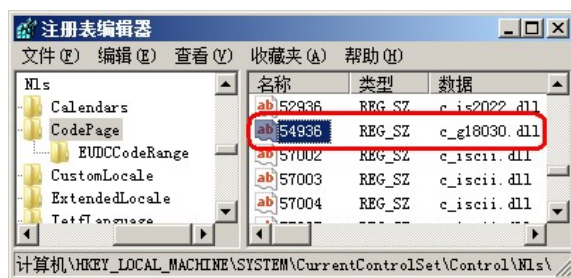


图1.5

在C:\Windows\System32和C:\Windows\SysWOW64目录下，均能找到c_g18030.dll这个文件。System32目录下是64位的，SysWOW64目录下是32位的。这个dll文件，导出了函数NlsDllCodePageTranslation。

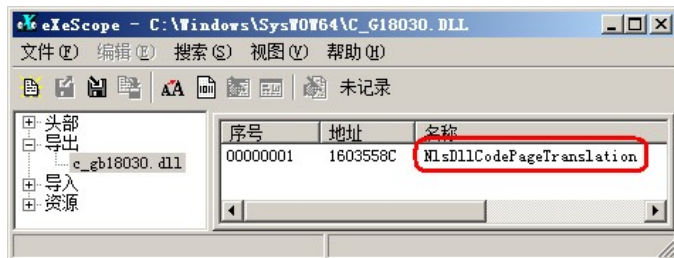


图1.6

也就是说，对于无法查表完成的编码转换。WideCharToMultiByte和MultiByteToWideChar会LoadLibrary该代码页对应的dll文件，然后调用该dll文件里的导出函数NlsDllCodePageTranslation，完成编码的转换工作。

第2章 区域

2.1 一个例子

首先看一个例子

```
#include <stdio.h>

void main()
{
    puts ("窄字符串");
    _putws(L"宽字符串");
}
```

运行结果如下：

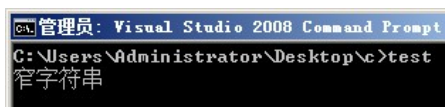


图2.1

为什么宽字符串没有被显示出来？因为调用C函数之前，没有设置C函数的代码页为GBK。为此，修改代码如下：

```
#include <stdio.h>

#include <locale.h>

void main()
{
    setlocale(LC_ALL, ".936"); //设置代码页为 GBK

    puts ( "窄字符串");
    _putws(L"宽字符串");
}
```

运行结果如下：

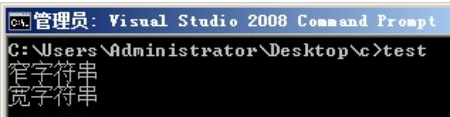


图2.2

2.2 setlocale

MSDN里关于setlocale函数的说明，第二个参数有些复杂，如下所示：

```
locale :: "lang[_country[.code_page]]"
| ".code_page"
| ""
| NULL
```

2.2.1 简单用法

用法	示例	说明
".code_page"	setlocale(LC_ALL, ".936");	设置代码页为936
""	setlocale(LC_ALL, "");	设置代码页为系统默认值 对于简体中文而言就是936
NULL	setlocale(LC_ALL, NULL);	获取设置，如： setlocale(LC_ALL, ""); 之前调用 setlocale(LC_ALL, NULL); 将 返回 "C" setlocale(LC_ALL, ""); 之后调用 setlocale(LC_ALL, NULL); 将 返回 "Chinese (Simplified)_People's

	Republic of China.936"
--	------------------------

2.2.2 复杂用法

setlocale(LC_ALL,"");之后调用setlocale(LC_ALL,NULL);将返回"Chinese (Simplified)_People's Republic of China.936"。这个返回值就是"lang[_country[.code_page]]"—下划线之前的是语言，下划线与小数点之间的是国家或地区，小数点之后的是代码页。

语言、国家或地区这两个参数该怎么填？可使用EnumSystemLocales函数枚举Windows系统的区域，然后使用GetLocaleInfo函数获得区域的属性。如下图所示：

LCID	LOCALE_SCOUNTRY	LOCALE_SENGLANGUAGE	LOCALE_SENGCOUNTRY	LOCALE_SNAME
0x0404(1028)	台湾	Chinese (Traditional)	Taiwan	950
0x0804(2052)	中华人民共和国	Chinese (Simplified)	People's Republic of China	936
0x0C04(3076)	香港特别行政区	Chinese (Traditional)	Hong Kong S.A.R.	950
0x1404(5124)	澳门特别行政区	Chinese (Traditional)	Macao S.A.R.	950

图2.3

上图第一列的LCID是EnumSystemLocales函数枚举出来的；LOCALE_SCOUNTRY、LOCALE_SENGLANGUAGE、LOCALE_SENGCOUNTRY……这些列是GetLocaleInfo函数获得的。

根据上图所示，使用setlocale函数设置台湾地区，可以这样设置：

```
setlocale(LC_ALL,"Chinese (Traditional)_Taiwan.950");
```

“语言_国家或地区.代码页”比较麻烦，可以使用缩写。缩写可由

GetLocaleInfo(...,LOCALE_SABBREVLANGNAME)获得，如下图所示：

LCID	LOCALE_SCOUNTRY	LOCALE_SABBREVLANGNAME
0x0404(1028)	台湾	CHT
0x0804(2052)	中华人民共和国	CHS
0x0C04(3076)	香港特别行政区	ZHH
0x1404(5124)	澳门特别行政区	ZHM

图2.4

根据上图所示，使用setlocale函数设置台湾地区，可以这样设置：

```
setlocale(LC_ALL,"CHT");
```

注意：

1、“Uzbek (Cyrillic)_Uzbekistan.1251”与“Uzbek (Latin)_Uzbekistan.1254”的缩写均为UZB。为防止混淆，请不要使用缩写；

2、更多的区域信息，可运行WinNLS程序获得，该程序已被笔者上传至git服务器，网址如下：

<https://github.com/hanford77/Exercise>

<https://git.oschina.net/hanford/Exercise>

2.3 #pragma setlocale

还是这段代码，在繁体中文操作系统下编译，会发生什么？

```
#include <stdio.h>

#include <locale.h>

void main()
{
    setlocale(LC_ALL,".936"); //设置代码页为 GBK
```



```
puts ( "窄字符串");  
  
_putws(L"宽字符串");  
  
}
```

首先,这段代码是在简体中文操作系统下编写的,并保存为ANSI编码格式。因此,"窄字符串"和"宽字符串"在源文件中被存储为多字节字符串,代码页为936 GBK。

编译器在编译"窄字符串"时,保持字符串的内容,因此"窄字符串"的编码仍为GBK编码;编译器在编译"宽字符串"时,需要将窄字符串转换为宽字符串。窄字符串的代码页本来是936的,结果在繁体中文操作系统下,VC++编译器会把该字符串的代码页当做950,然后转换为宽字符串。结果就会产生乱码了。

为此,可添加一行代码,如下所示:

```
#include <stdio.h>  
  
#include <locale.h>  
  
void main()  
{  
  
setlocale(LC_ALL,".936");//设置代码页为 GBK  
  
puts ( "窄字符串");  
  
#pragma setlocale(".936")  
  
_putws(L"宽字符串");  
  
}
```

#pragma setlocale(".936")的含义就是:让编译器执行一下函数setlocale(...,".936");将代码页切换为936。这样,在将"宽字符串"转换为宽字符串时,就不会产生乱码了。

#pragma setlocale的参数,可以完全按照setlocale函数第二个参数的格式进行填写。

2.4 rc 文件

资源文件(*.rc)同样需要设置区域和代码页,具体如下

```
LANGUAGE LANG_CHINESE, SUBLANG_CHINESE_SIMPLIFIED  
  
#pragma code_page(936)
```

LANGUAGE LANG_CHINESE, SUBLANG_CHINESE_SIMPLIFIED指定区域的LCID为0x0804(十进制的2052)。参考图2.4,就是设置区域为"中华人民共和国"。

#pragma code_page(936)就是设置代码页为936。具体就是:从这一行开始,所有的字符串均是GBK编码。

注意:VC++.NET的rc文件可以保存为Unicode(UTF-16LE)格式。此时,设置区域和代码页似乎不是必需的了。

2.5 排序

LCID里还包含了排序信息。根据这个排序设置,可使用CompareString函数比较两个字符串。

将有排序信息的LCID设置给ComboBox、ListBox,则会影响这些控件的排序功能(其内部应该是调用了CompareString函数),具体请参考WinNLS工程。

文档下载

分类: 字符串

好文要顶

关注我

收藏该文

微信分

享



hanford

粉丝 - 41 关注 - 0

+加关注

1

0

[升级成为会员](#)

« 上一篇: [UTF-7编码](#)

» 下一篇: [VC++ 中使用 std::string 转换字符串编码](#)

posted @ 2016-11-29 14:01 hanford 阅读(7719) 评论(2) 收藏 举报

博客园 © 2004-2025